

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS
COMPUTACIONALES



Práctica 4.- Uso de las Expresiones Regulares y
Extensión de Software para Convertir
Expresiones Regulares

Presenta:

Juárez Velázquez Erick Daniel 2025630052

Placencia Murguía Juan Emilio 2025630024

Profesor:

Gabriel Hurtado Avilés

Asignatura:

Teoría de la Computación

Grupo:

4CM4

Ejercicio 1

1. Investigar y documentar los siguientes aspectos de las expresiones regulares:

- **Definición formal de lenguaje regular:**

Un lenguaje se considera regular si puede ser descrito por una expresión regular o aceptado por un autómata finito. De manera formal, dado un alfabeto, el conjunto de lenguajes regulares se define de forma recursiva:

- ❖ $\emptyset$ (el lenguaje vacío) es un lenguaje regular.
- ❖ $\{\lambda\}$ (el lenguaje que contiene únicamente la cadena vacía) es un lenguaje regular.
- ❖ $\forall a \in \Sigma$, el conjunto $\{a\}$ es un lenguaje regular.
- ❖ Si A y B son lenguajes regulares, entonces las operaciones aplicadas sobre ellos (unión $\cup$, concatenación y cerradura de Kleene $*$)

- **Definición formal y componentes básicos de las expresiones regulares.**

Las expresiones regulares (ER) son un equivalente algebraico de los autómatas finitos y proporcionan una manera declarativa de describir y simplificar los patrones que forman un lenguaje regular.

Sus componentes básicos o expresiones primitivas sobre un alfabeto Σ son:

- ❖ $\emptyset$: Expresión que representa el lenguaje vacío.
- ❖ λ : Expresión que representa la cadena vacía.
- ❖ a : Expresión para cada símbolo individual donde $a \in \Sigma$

- **Operaciones fundamentales en expresiones regulares**

Las expresiones regulares más complejas se construyen combinando las expresiones primitivas mediante tres operaciones fundamentales:

- ❖ Unión: Usualmente representada con el símbolo $+$ o $\cup$. Si r y s son expresiones regulares, $r \cup s$ genera el lenguaje que resulta de unir los lenguajes descritos por r y s .
- ❖ Concatenación: Representada con un punto $\cdot$ o simplemente mediante la yuxtaposición de variables (rs). Combina las cadenas tomando primero una del lenguaje r seguida inmediatamente por una del lenguaje s .
- ❖ Cerradura de Kleene: Denotado por un asterisco $*$. La expresión r^* indica la repetición de la expresión base cero o más veces.

- **Jerarquía de Chomsky y ubicación de los lenguajes regulares**

La Jerarquía de Chomsky es un sistema que clasifica las gramáticas formales y los lenguajes que generan en cuatro niveles (del Tipo 0 al Tipo 3), dependiendo de sus restricciones estructurales.

Los lenguajes regulares se ubican en el **Tipo 3**. Esta es la categoría más restrictiva de toda la jerarquía. Estos lenguajes son generados exclusivamente por gramáticas regulares (ya sean lineales por la derecha o por la izquierda) y su máquina aceptadora correspondiente es el autómata finito. (Ver Figura 1)

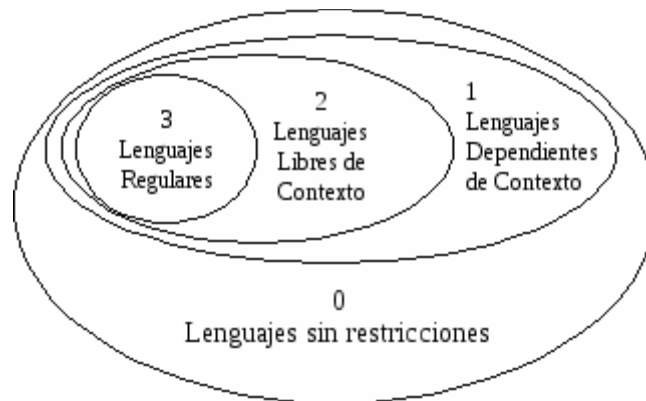


Figura 1

- **Equivalencia entre expresiones regulares y autómatas finitos (Teorema de Kleene).** Propuesto por el matemático Stephen Cole Kleene, este teorema es uno de los pilares de la teoría de autómatas. Su declaración formal establece que *un lenguaje es regular si y solo si es aceptado por un autómata finito*.

Para entender su impacto real, podemos desglosarlo en los siguientes conceptos clave:

- ❖ **Equivalencia de Poder Computacional:** El teorema demuestra matemáticamente que diferentes enfoques para definir o reconocer lenguajes tienen exactamente el mismo poder descriptivo y computacional. En la teoría formal, esto se suele denotar como:

$$AFD \equiv AFN \equiv AFN - \lambda \equiv ER$$

Esto significa que lo que puedes describir con una Expresión Regular (ER), también lo puedes reconocer con un Autómata Finito Determinista (AFD), con un No Determinista (AFN) o con uno que tenga transiciones vacías (AFN- λ). Ninguno más capaz de reconocer más lenguajes que los otros.

- ❖ **Bidireccionalidad:** El Teorema de Kleene no solo es una declaración abstracta, sino que garantiza que existen algoritmos sistemáticos para transformar un modelo en otro:

- **De Expresión Regular a Autómata:** Puedes tomar cualquier expresión regular y construir un autómata que reconozca exactamente el mismo lenguaje de cadenas.
- **De Autómata a Expresión Regular:** De manera inversa, si tienes un autómata finito (como un AFD), existe la garantía de que puedes extraer una expresión regular que describa sus caminos desde el estado inicial hasta los estados finales de aceptación. Existen varios métodos teóricos para lograr esto, como el algoritmo de eliminación de estados o el método de derivadas de Brzozowski.

2. Investigar y documentar al menos cinco aplicaciones prácticas de las expresiones regulares en la computación moderna, como:

- **Procesamiento de texto y búsqueda de patrones.**

Las expresiones regulares son la herramienta por excelencia para buscar, encontrar y extraer información específica dentro de grandes volúmenes de texto, como documentos o bases de datos.

Ejemplo de ER:

`\b\d{4}-\d{2}-\d{2}\b`

Cómo funciona: Este patrón busca fechas en formato YYYY-MM-DD (como 2026-04-30). `\b` asegura que la fecha sea una palabra independiente; `\d{4}` busca exactamente cuatro números para el año; `-` busca el guion literal; y `\d{2}` busca dos números para el mes y el día, respectivamente. (Ver Figura 1)

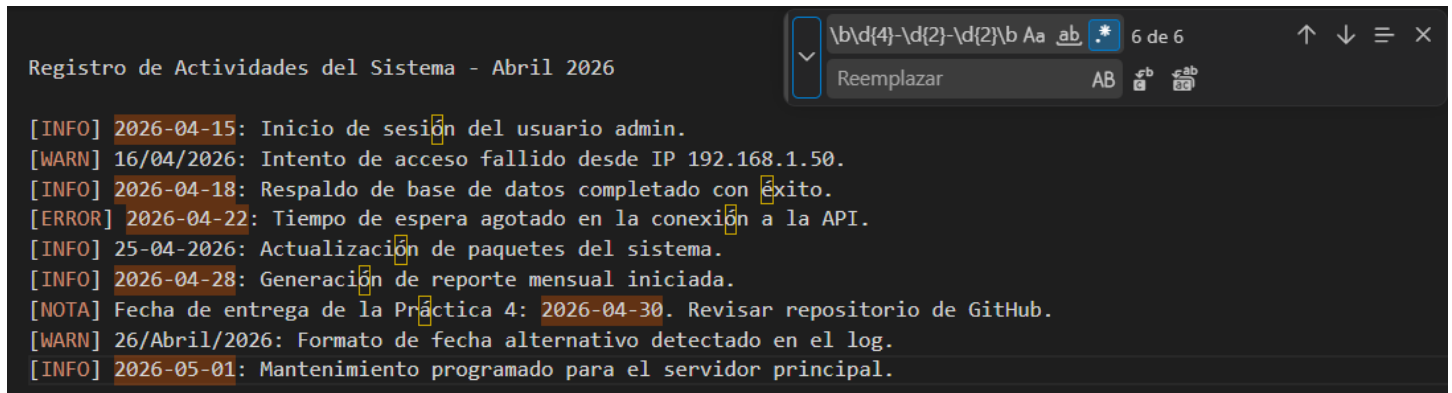


Figura 1

- **Análisis léxico en compiladores.**

En la primera fase de la compilación de un programa, el analizador léxico o "scanner" lee el código fuente y lo divide en componentes básicos llamados "tokens" (palabras reservadas, variables, operadores). Las expresiones regulares definen las reglas para reconocer cada uno de estos tokens.

Ejemplo de ER: `^[a-zA-Z_][a-zA-Z0-9_]*$`

Cómo funciona: Esta expresión valida el nombre de una variable (identificador) en lenguajes como C o Java. $^{[a - zA - Z]}$ indica que el nombre debe empezar obligatoriamente con una letra o un guion bajo. $[a - zA - Z0 - 9_]*\$$ permite que el resto del nombre contenga cero o más letras, números o guiones bajos, hasta el final de la cadena.(Ver Figura 2)

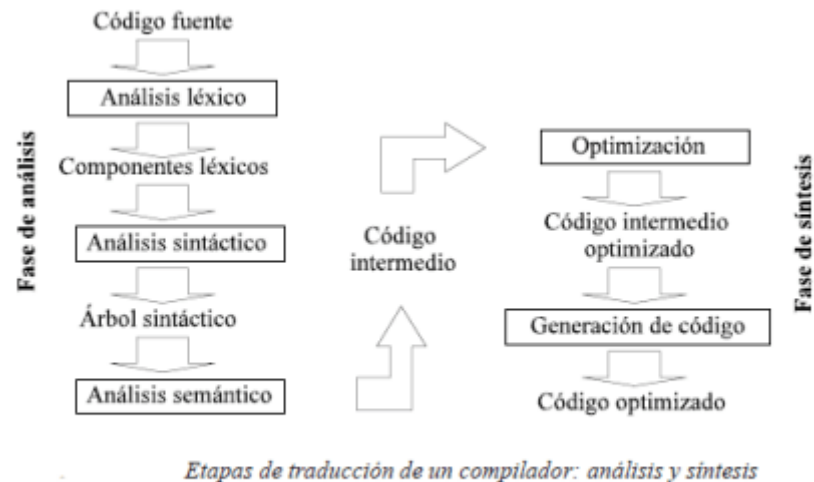


Figura 2

- **Validación de datos en formularios web.**

Cuando te registras en un sitio web, el sistema necesita asegurarse de que la información introducida tenga el formato correcto antes de enviarla a la base de datos (por ejemplo, que un correo no tenga espacios o que una contraseña sea segura).

Ejemplo de ER: $^{[a - zA - Z0 - 9._\% + -]} + @[a - zA - Z0 - 9.-] + \. [a - zA - Z]{2,} \$$

Cómo funciona: Valida una dirección de correo electrónico básica. La primera parte $[a - zA - Z0 - 9._\% + -] +$ acepta los caracteres válidos del nombre de usuario antes del arroba @. Luego, $[a - zA - Z0 - 9.-] +$ acepta el nombre del dominio, seguido de un punto literal \. . Finalmente, $[a - zA - Z]{2,} \$$ asegura que la extensión final (como .com o .mx) tenga al menos dos letras. (Ver Figura 3)

La imagen muestra un formulario web con el título "Correo electrónico o teléfono". El campo de entrada contiene el texto "teoría_de_la_computacion@semestre.2026-2". Debajo del campo, hay un mensaje de error que dice: "Ingresa un número de teléfono o correo electrónico válidos".

Figura 3

- **Filtrado y manipulación de datos.**

Los administradores de sistemas y analistas de datos usan expresiones regulares para limpiar y extraer información útil de archivos de registro (logs) que contienen miles de líneas de texto generadas por servidores o aplicaciones.

Ejemplo de ER: `\b(?:\d{1,3}\.){3}\d{1,3}\b`

Cómo funciona: Este patrón extrae direcciones IP (IPv4) de un archivo de texto desordenado. `\d{1,3}` busca de uno a tres dígitos (por ejemplo, "192"), seguido de un punto literal `\.`. Todo esto se repite tres veces gracias al cuantificador `{3}`. Finalmente, se busca el último bloque de uno a tres dígitos. (Ver Figura 4)

```
root@vps40618123:/# grep -i command grep
How to Use Grep Command in Linux? + some Useful Grep Examples
In this post, we will teach you how to use the grep command in Linux, and reinforce the learning with a useful example for your daily workflow.
1. How to Use the grep Command?
The grep command belonging to the Unix family is one of the most versatile and useful tools available. It looks in a text file for a pattern that we define. In other words, with grep you can search for a word or pattern and the line or lines that have it will be printed.
At first sight, it can be a command of little utility, however, sysadmins that handle many services with various configuration files, use it to query or search for specific lines within those files.
```

Figura 4

- **Automatización de tareas en sistemas operativos**

Instrucción de sustitución para cambiar masivamente la extensión de archivos .jpeg a .jpg.

Expresión Regular:

`s/\.jpeg$/.jpg/`

La variable denota la función de sustitución o mapeo. Se divide en dos argumentos separados por la diagonal `.`. El primer argumento busca el patrón literal justo al final del elemento (denotado por `)`, y el segundo argumento es el valor de reemplazo que tomará su lugar. (Ver Figura 5)

```
erick@ubuntu:~/practica4_archivos$ ls
reporte_01_enero.txt  reporte_02_febrero.txt  reporte_03_marzo.txt

erick@ubuntu:~/practica4_archivos$ rename 's/^reporte_(\d{2})_([a-z]+)\.txt$/documento_$1.bak/' *.txt

erick@ubuntu:~/practica4_archivos$ ls
documento_01.bak  documento_02.bak  documento_03.bak
erick@ubuntu:~/practica4_archivos$
```

Figura 5

Fuentes de información

- Carrión Viramontes, J. E. (2023). *Teoría de la computación*. Instituto Tecnológico de Morelia (ITM). Recuperado de <https://dsc.itmorelia.edu.mx/~jcolivares/nlp/libros/teoriaComputacion.pdf>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2002). *Introducción a la teoría de autómatas, lenguajes y computación* (2.^a ed.). Addison-Wesley / Pearson.
- Mozilla Developer Network. (2026). *Regular expressions - JavaScript*. MDN Web Docs. Recuperado de https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions
- Universidad Autónoma de la Ciudad de México. (s.f.). *Programa de estudios: Teoría de la computación*. UACM. Recuperado de https://uacm.edu.mx/Portals/0/OfertaAcademica/CCyT/Programas/CS/Software/5/Teoria_Computacion.pdf